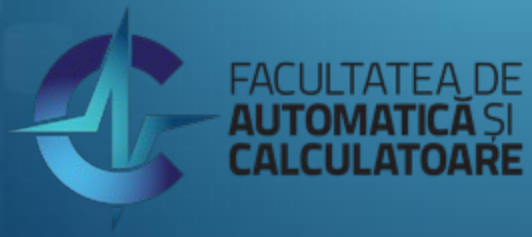


Algoritmi Paraleli și Distribuiți
Lucru cu semafoare

Radu-Ioan CIOBANU
radu.ciobanu@upb.ro





Sincronizare: Notiuni introductive

Dezvoltarea algoritmilor folosind variabile partajate (MIMD)

- Sincronizarea: excluderea mutuală şi sincronizarea condiţionată.
- Câteva rezultate notabile:
  - mutex
  - semafoare
    - regiuni critice
  - bariere
- Câteva dintre problemele *clasice*:
  - producători-consumatori
  - problema filozofilor
  - cititori-scriitori
  - problema bărbierului

Sectiuni critice

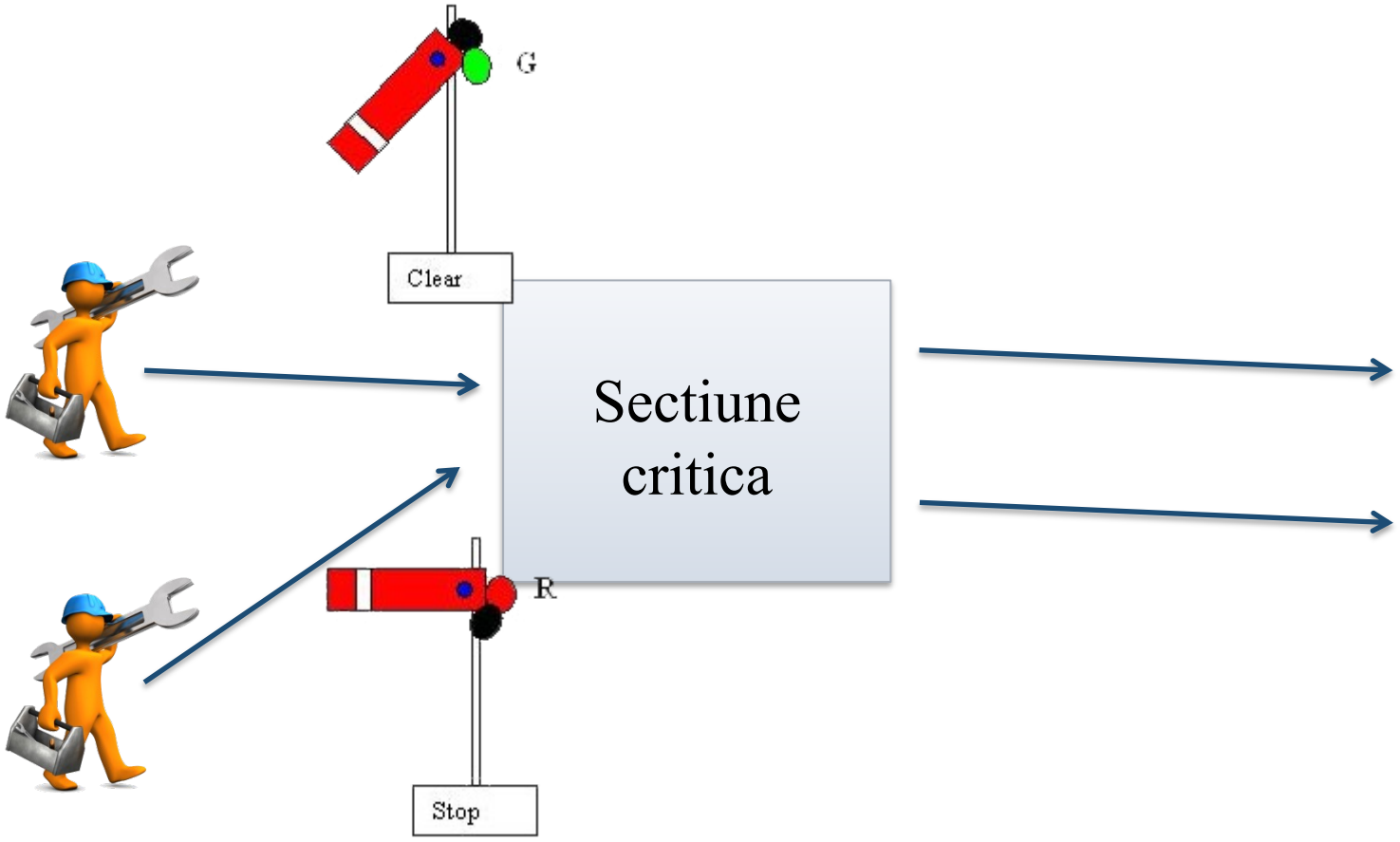
- Problema: fiecare proces P(i) al unei colectii de procese P(i:1..n) executa ciclic o sectiune critica, in care are acces exclusiv la anumite resurse partajate, urmata de o sectiune necritica in care foloseste doar resurse locale.

- Semafor = tip special de variabile partajate manipulate prin 2 operații atomice: P și V *(Dijkstra, 1965)*
  - Asigura excluderea mutuala intre procesele care acceseaza sectiunea critica.

Secțiuni critice

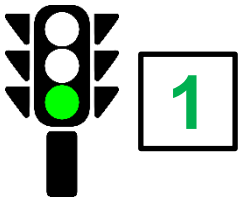
```
sem mutex= 1;
process P[i=1 to n] {
    while (true) {
      P(mutex); /* acaparare secțiune critică */
      Secțiune critică;
      V(mutex); /* eliberare secțiune critică */
      Secțiune necritică
    }
}
```

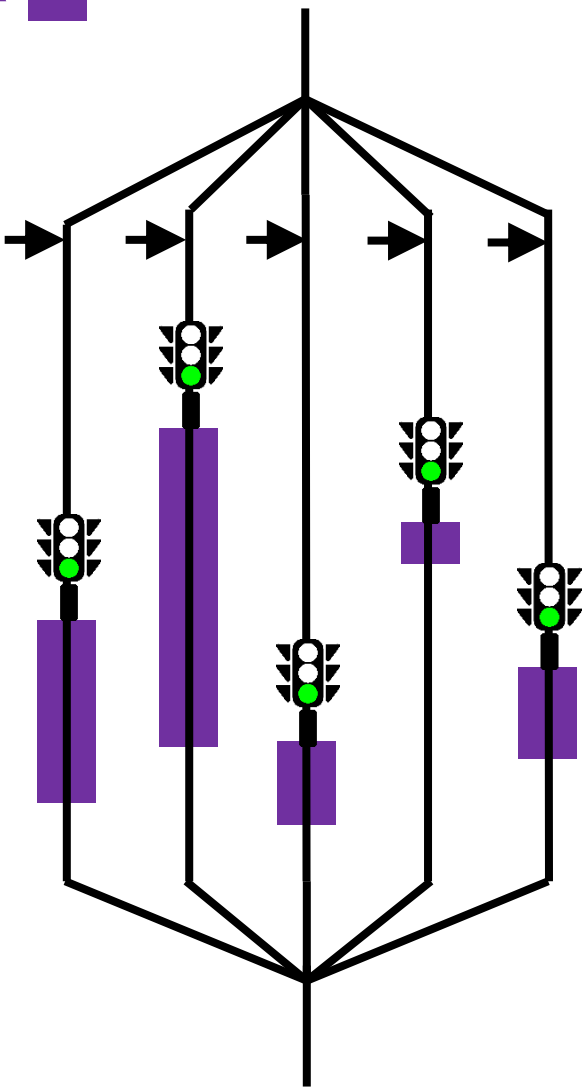
Exemplu executie…



Zonă Critică

P() sau Proberen
V() sau Verhogen
 - Dijsktra

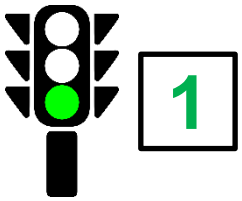


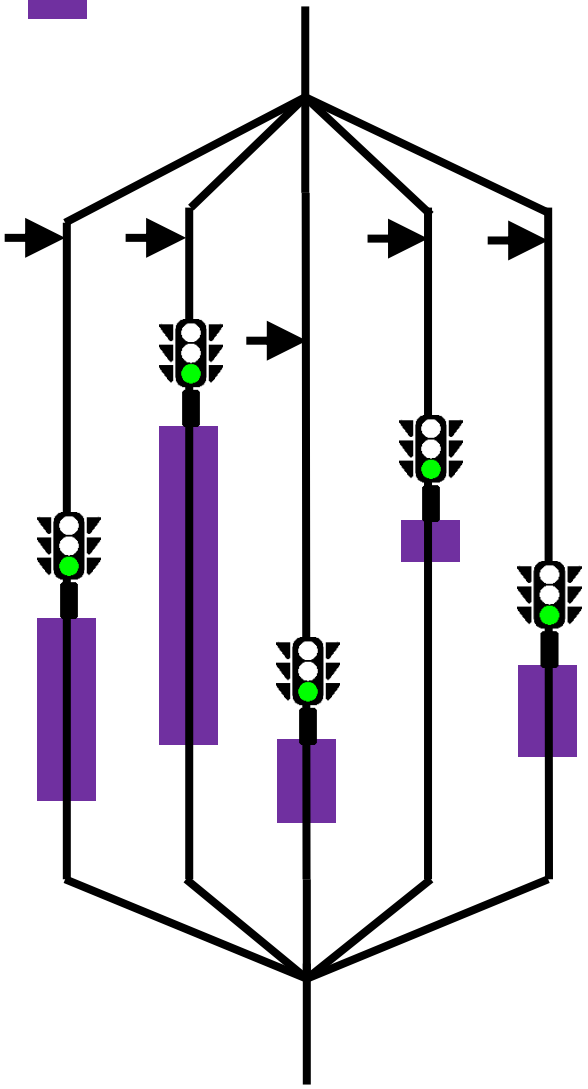


Doar un thread poate fi în zona critică la un moment dat

Zonă Critică

P() sau Proberen
V() sau Verhogen
 - Dijsktra

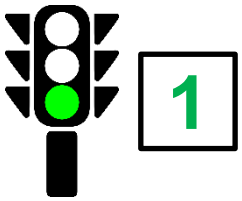


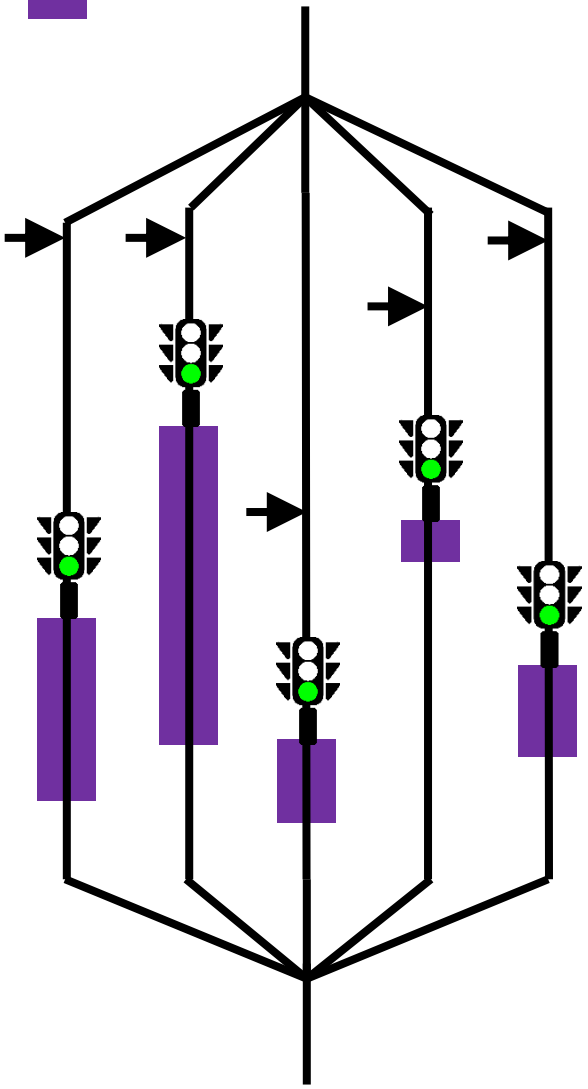


Doar un thread poate fi în zona critică la un moment dat

Zonă Critică

P() sau Proberen
V() sau Verhogen
 - Dijsktra

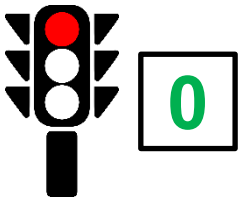


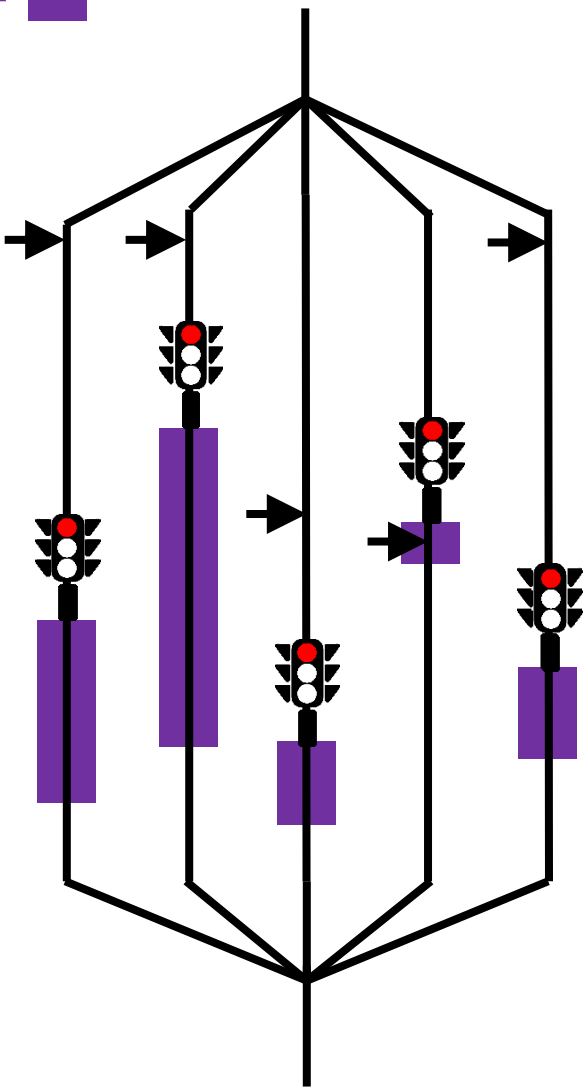


Doar un thread poate fi în zona critică la un moment dat

Zonă Critică

P() sau Proberen
V() sau Verhogen
 - Dijsktra

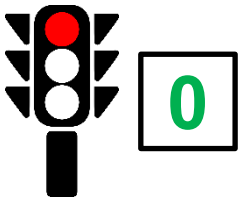


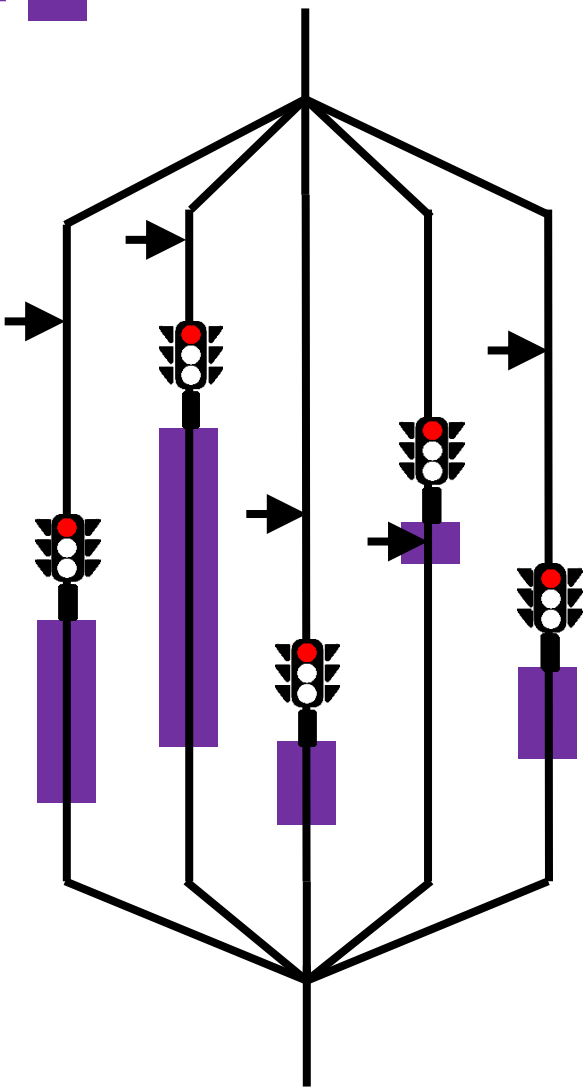


Doar un thread poate fi în zona critică la un moment dat

Zonă Critică

P() sau Proberen
V() sau Verhogen
 - Dijsktra

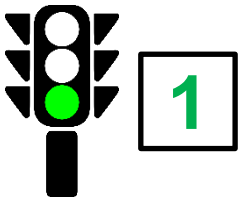


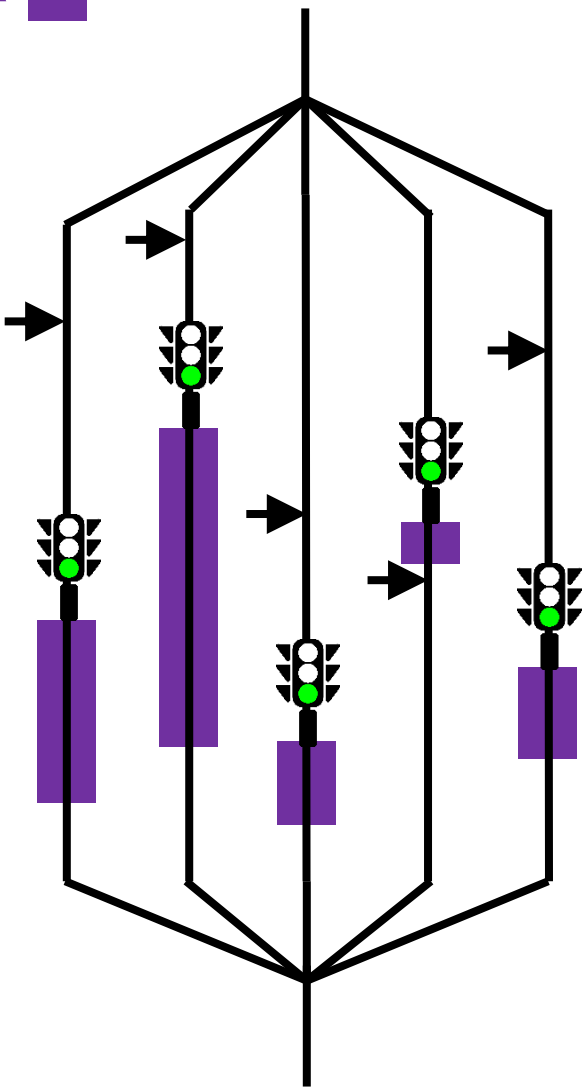


Doar un thread poate fi în zona critică la un moment dat

Zonă Critică

P() sau Proberen
V() sau Verhogen
 - Dijsktra

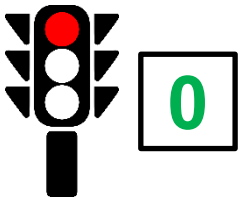


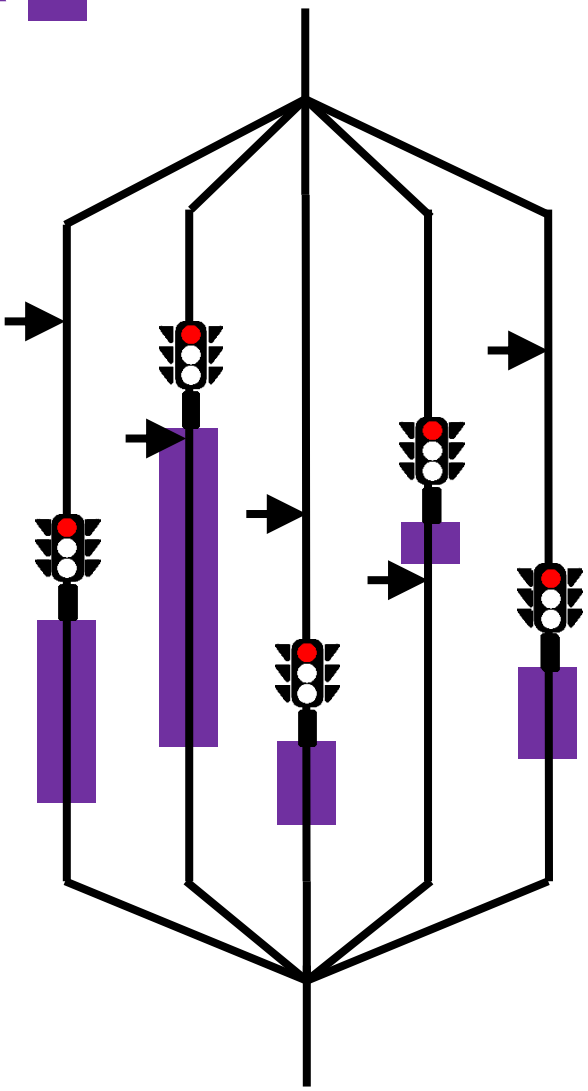


Doar un thread poate fi în zona critică la un moment dat

Zonă Critică

P() sau Proberen
V() sau Verhogen
 - Dijsktra

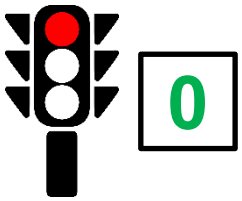


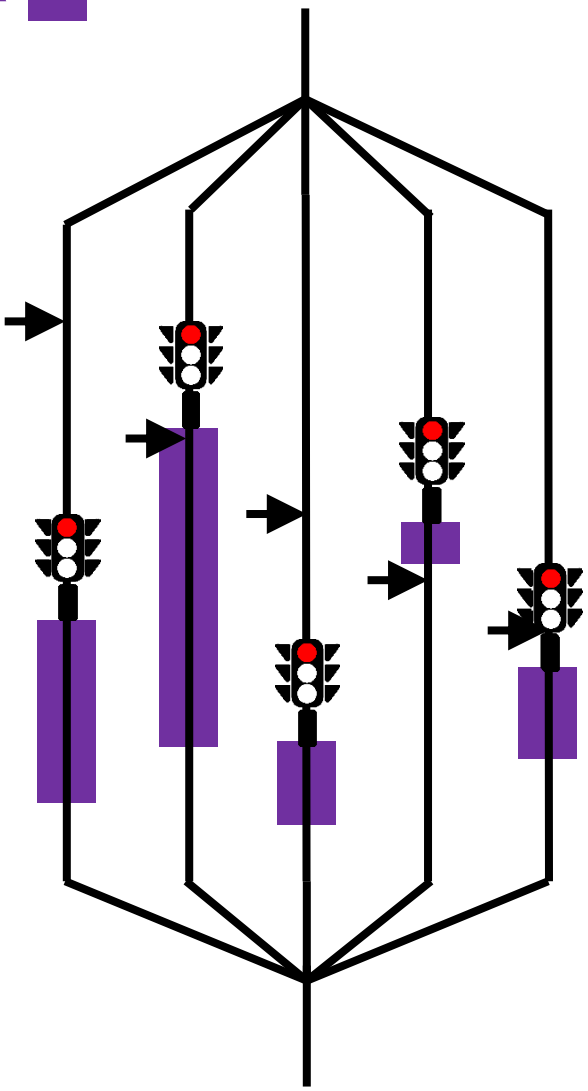


Doar un thread poate fi în zona critică la un moment dat

Zonă Critică

P() sau Proberen
V() sau Verhogen
 - Dijsktra

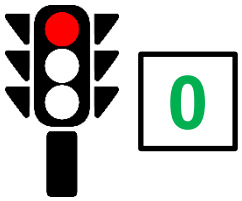


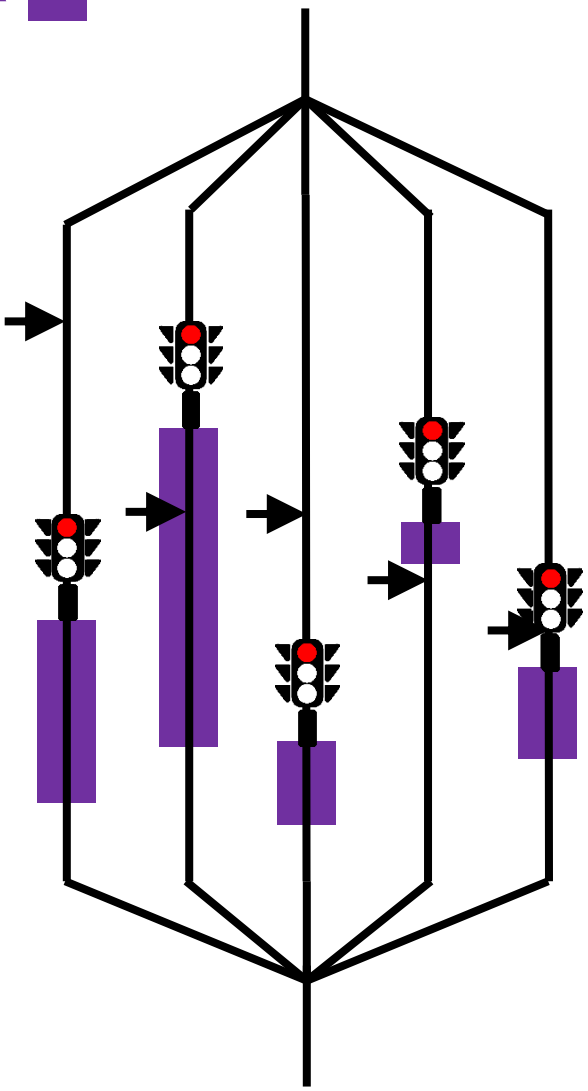


Doar un thread poate fi în zona critică la un moment dat

Zonă Critică

P() sau Proberen
V() sau Verhogen
 - Dijsktra

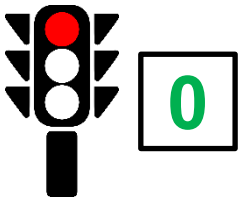


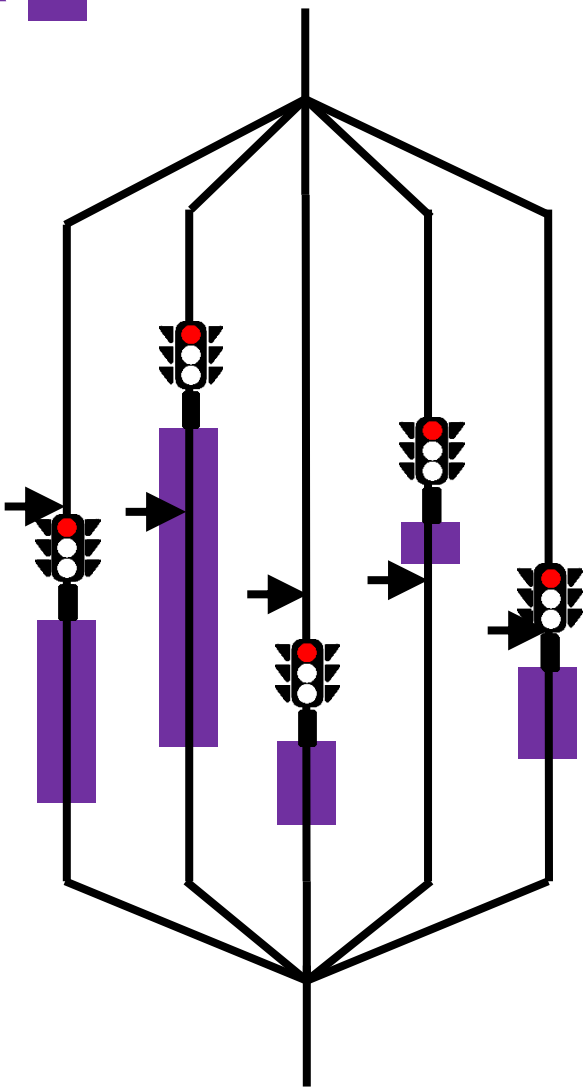


Doar un thread poate fi în zona critică la un moment dat

Zonă Critică

P() sau Proberen
V() sau Verhogen
 - Dijsktra

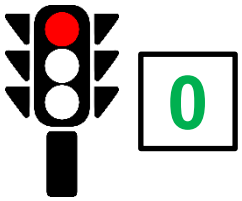


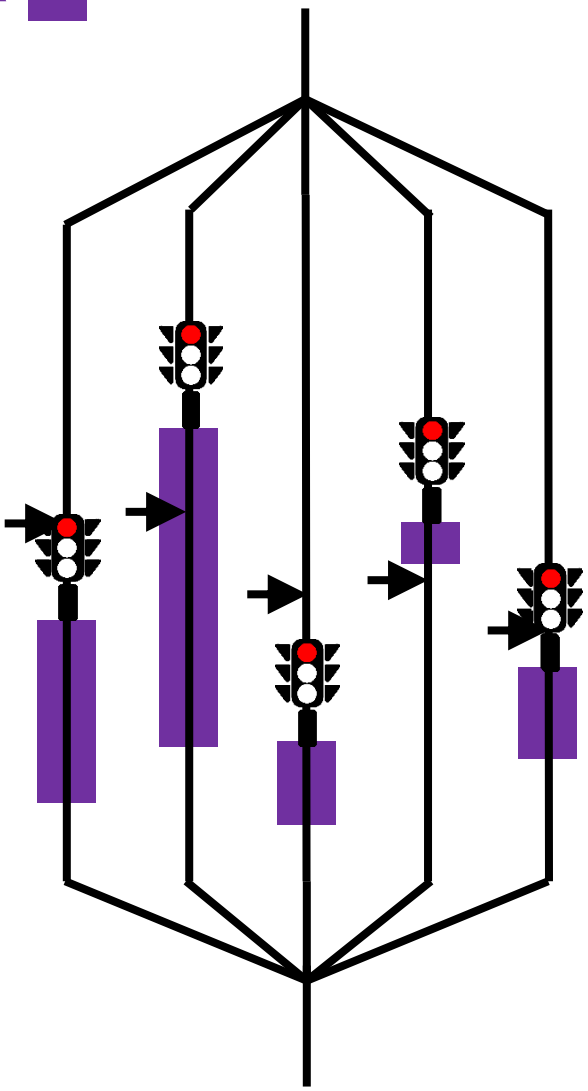


Doar un thread poate fi în zona critică la un moment dat

Zonă Critică

P() sau Proberen
V() sau Verhogen
 - Dijsktra

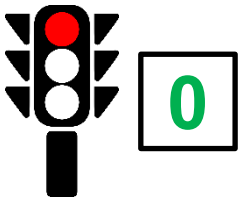


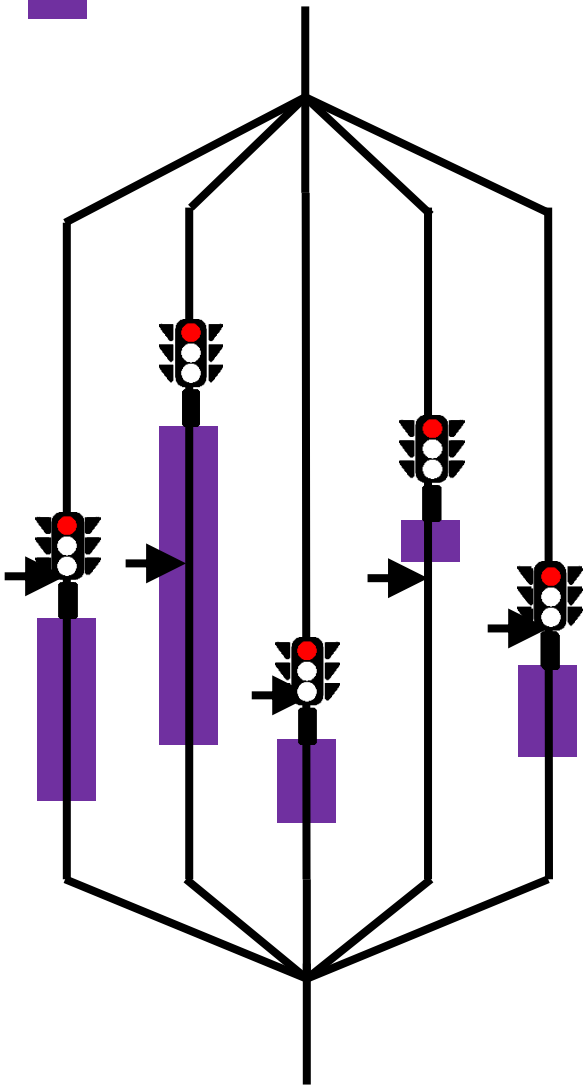


Doar un thread poate fi în zona critică la un moment dat

Zonă Critică

P() sau Proberen
V() sau Verhogen
 - Dijsktra

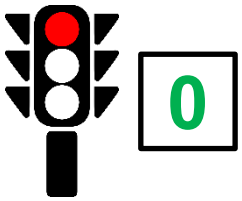


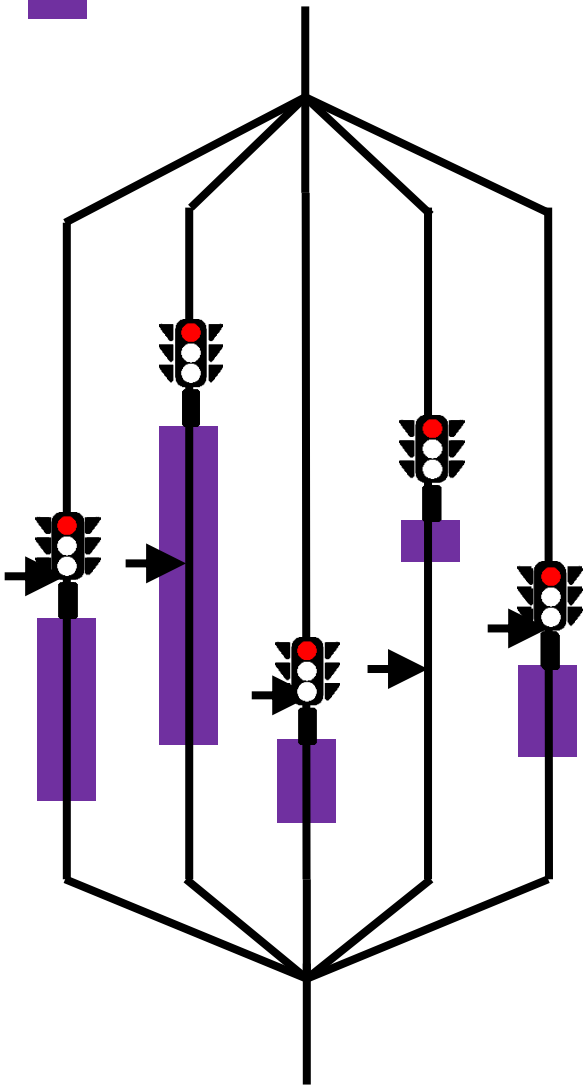


Doar un thread poate fi în zona critică la un moment dat

Zonă Critică

P() sau Proberen
V() sau Verhogen
 - Dijsktra

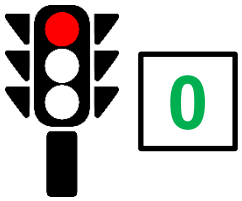


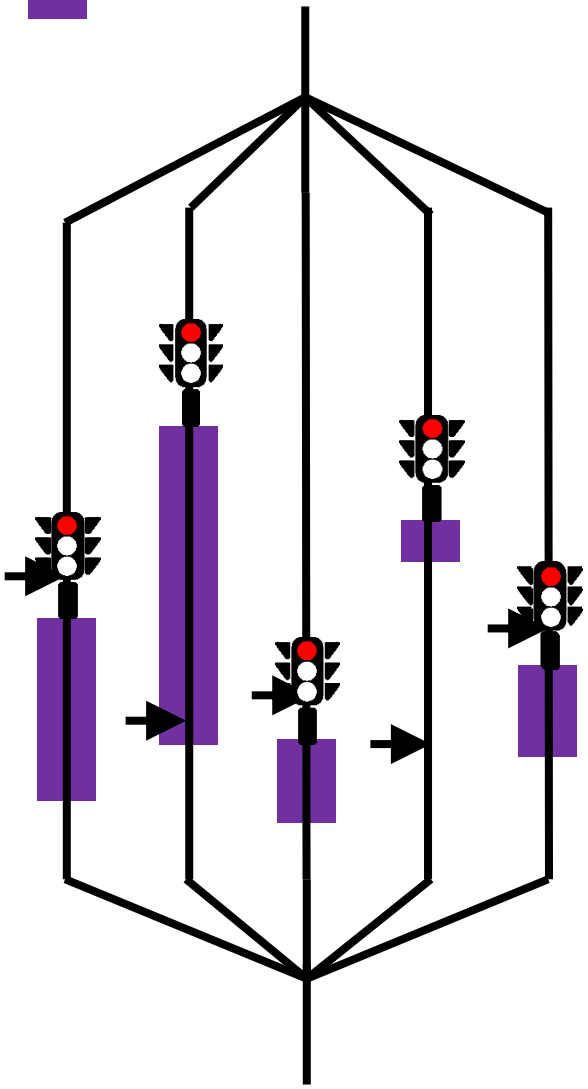


Doar un thread poate fi în zona critică la un moment dat

Zonă Critică

P() sau Proberen
V() sau Verhogen
 - Dijsktra



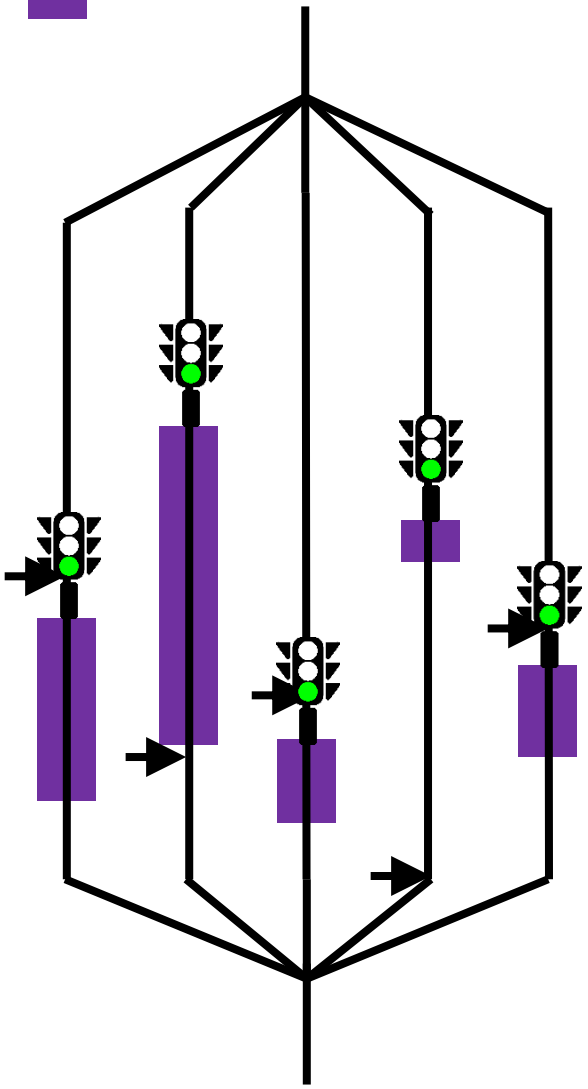


Doar un thread poate fi în zona critică la un moment dat

Zonă Critică

P() sau Proberen
V() sau Verhogen
 - Dijsktra



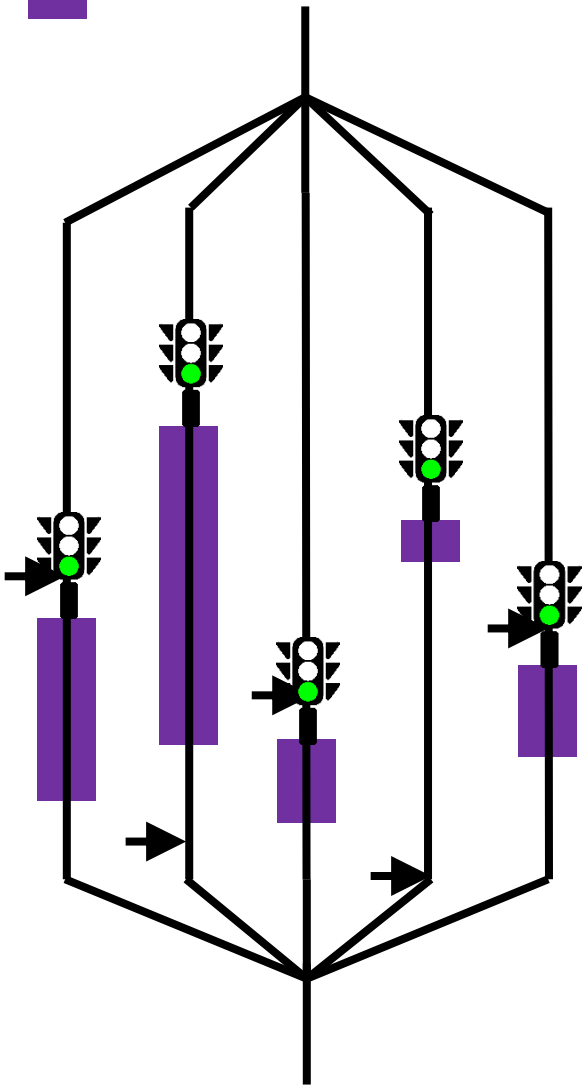


Doar un thread poate fi în zona critică la un moment dat

Zonă Critică

P() sau Proberen
V() sau Verhogen
- Dijsktra

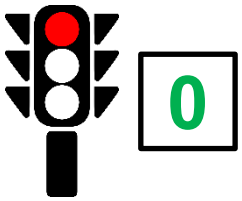


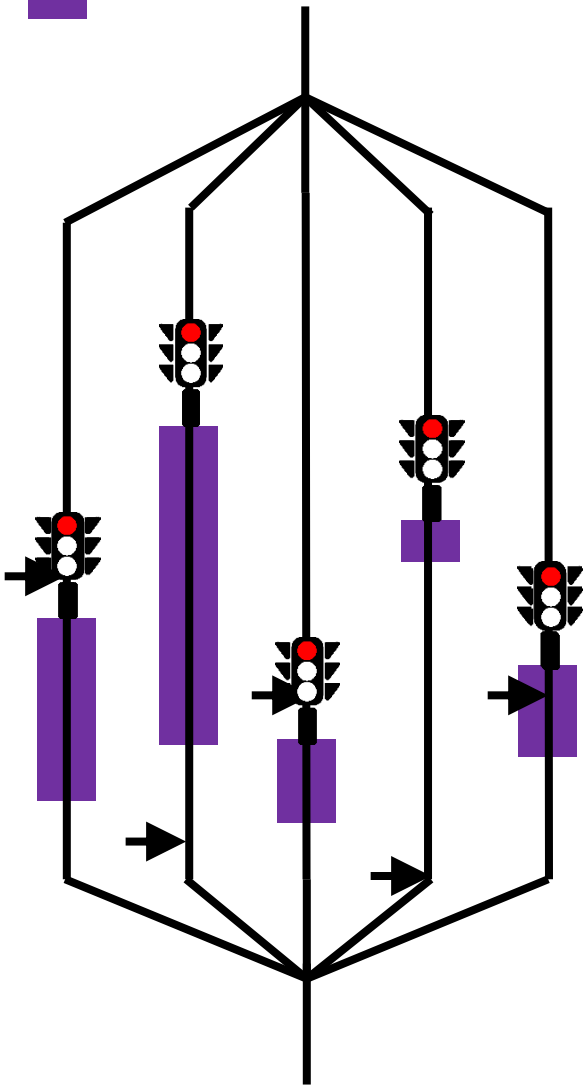


Doar un thread poate fi în zona critică la un moment dat

Zonă Critică

P() sau Proberen
V() sau Verhogen
 - Dijsktra





Doar un thread poate fi în zona critică la un moment dat

Producator-consumator

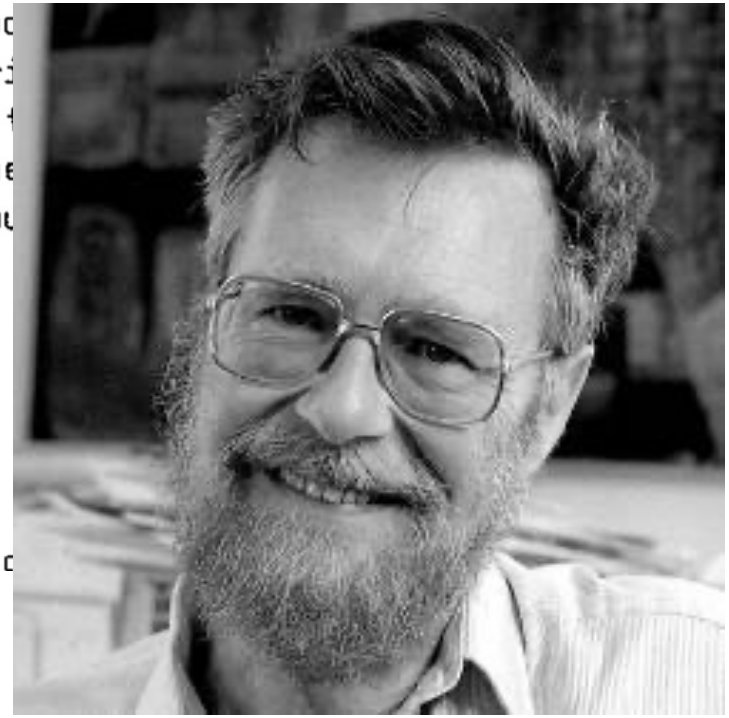
Producer - Consumer

EWD209 - 0

EWD209.html

A Constructive Approach to the Problem of Program Correctness.

Summary. As an alternative to methods by which the correctness of given programs can be established a posteriori, this paper propo of program generation such as to produce a priori is treated to show the form that such a control t comes from the field of parallel programming; the is representative for the way in which a whole mu actually been constructed.

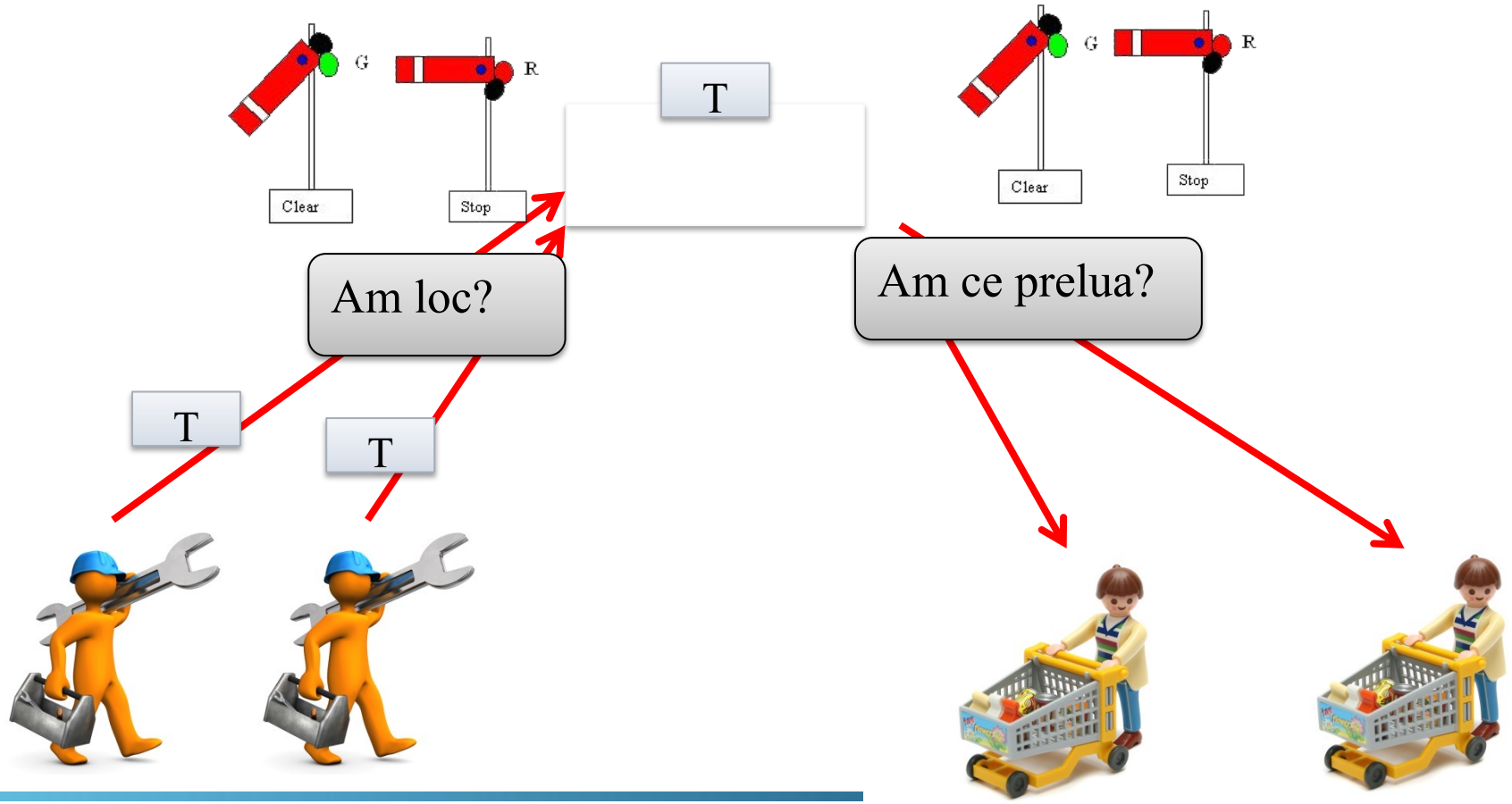


Introduction.

The more ambitious we become in our machine becomes the problem of program correctness. The c

Producători și consumatori

- Problema
  - Se consideră M producători și N consumatori care comunică printr-un singur buffer partajat.



Producători și consumatori

```
typeT buf; /* T - tipul datelor */
sem gol= 1, plin= 0; /* sem binare */
process Producător[i=1 to M]{
    typeT v;
    while(true) {
      v = produce();
      P(gol);
      buf = v;
      V(plin);
    }
}
```

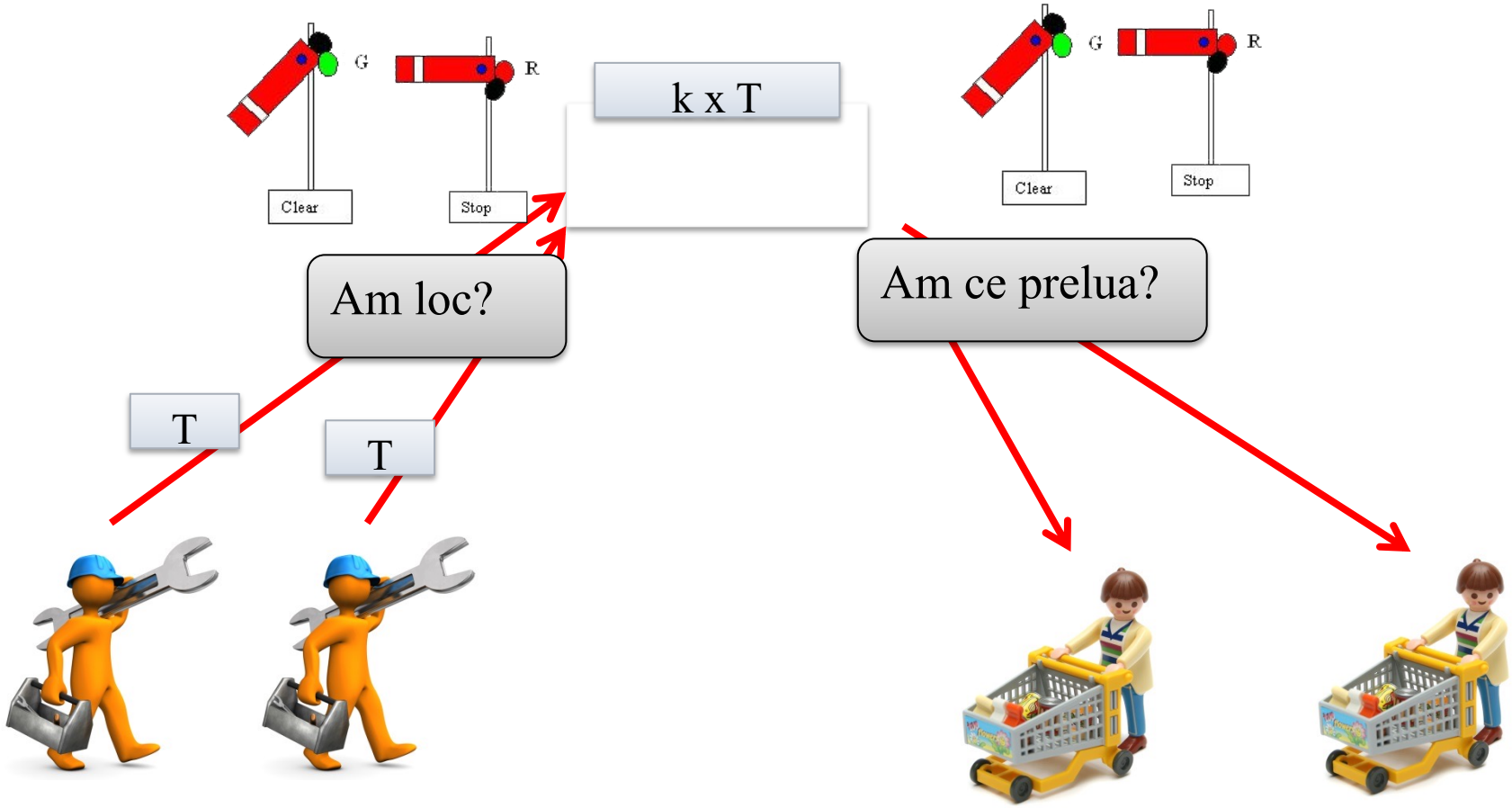
```
process Consumator[i=1 to N]{
    typeT w;
    while(true) {
      P(plin);
      w = buf;
      V(gol);
      consumă(w);
    }
}
```

semnalizare

Comunicarea producător și consumator prin tampon limitat

```
typeT buf[1:k];
sem gol = k, plin = 0;  /* semafoare generale */
process Producător {
      typeT v;
      int ultim= 1;
      while (true) {
        v = produce();
        P(gol);                      /* există locuri goale? */
        buf[ultim] = v;
        ultim = ultim mod k + 1; /* circular */
        V(plin);
      } }
process Consumator {
      typeT w;
      int prim= 1;
      while (true) {
        P(plin);                     /* există valori în buffer? */
        w = buf[prim]; prim = prim mod k + 1;
        V(gol);
        consumă(w);
      }
}
```

Comunicarea producător și consumator prin tampon limitat



Mai mulți producători și mai mulți consumatori

```
typeT buf[1:k];
int prim = 1, ultim = 1;
sem gol = k, plin = 0;
sem mutexP = 1, mutexC = 1;
```

```
process Producător[i=1 to M]{
    typeT v;
    while (true) {
      v = produce();
      P(gol);
       P(mutexP);
       buf[ultim] = v;
       ultim = ultim mod k + 1;
       V(mutexP);
      V(plin);
    }
}
```

```
process Consumator[i=1 to N]{
    typeT w;
    while (true) {
P(plin);
      P(mutexC);
      w = buf[prim];
      prim = prim mod k + 1;
     V(mutexC);
    V(gol);
    consumă(w);
    }
}
```

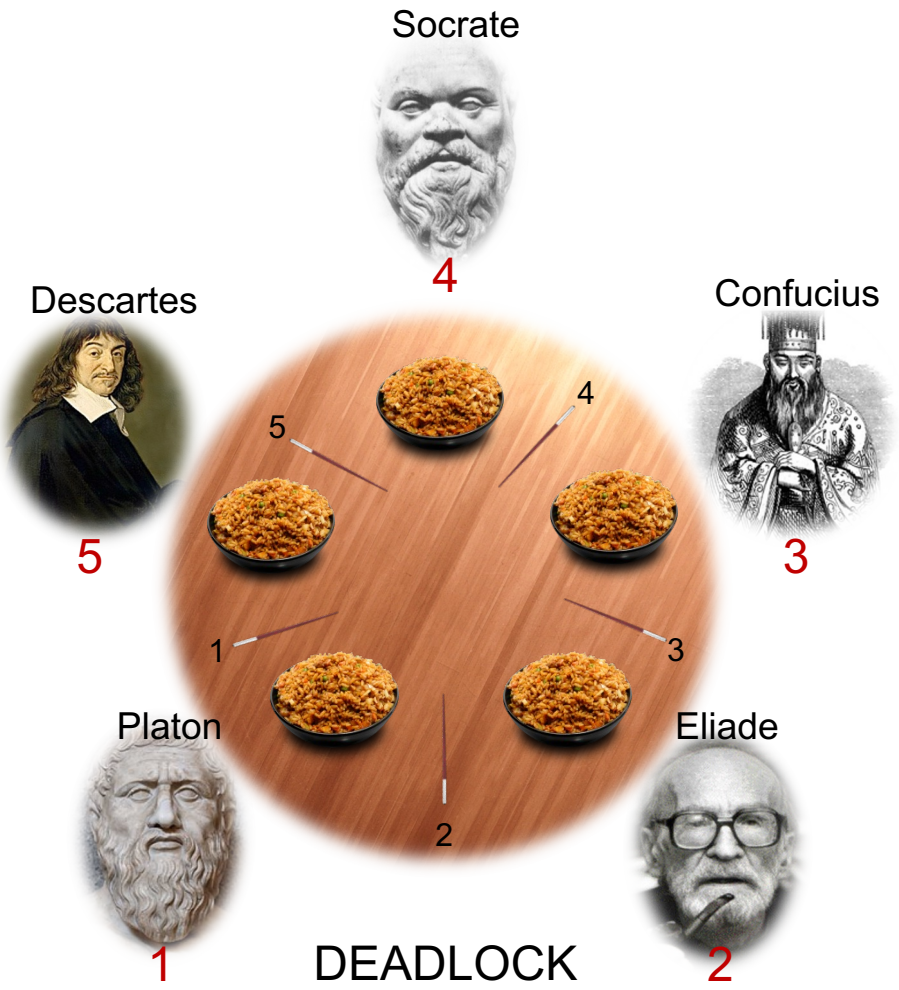
secțiune critică

semnalizare

Problema filozofilor

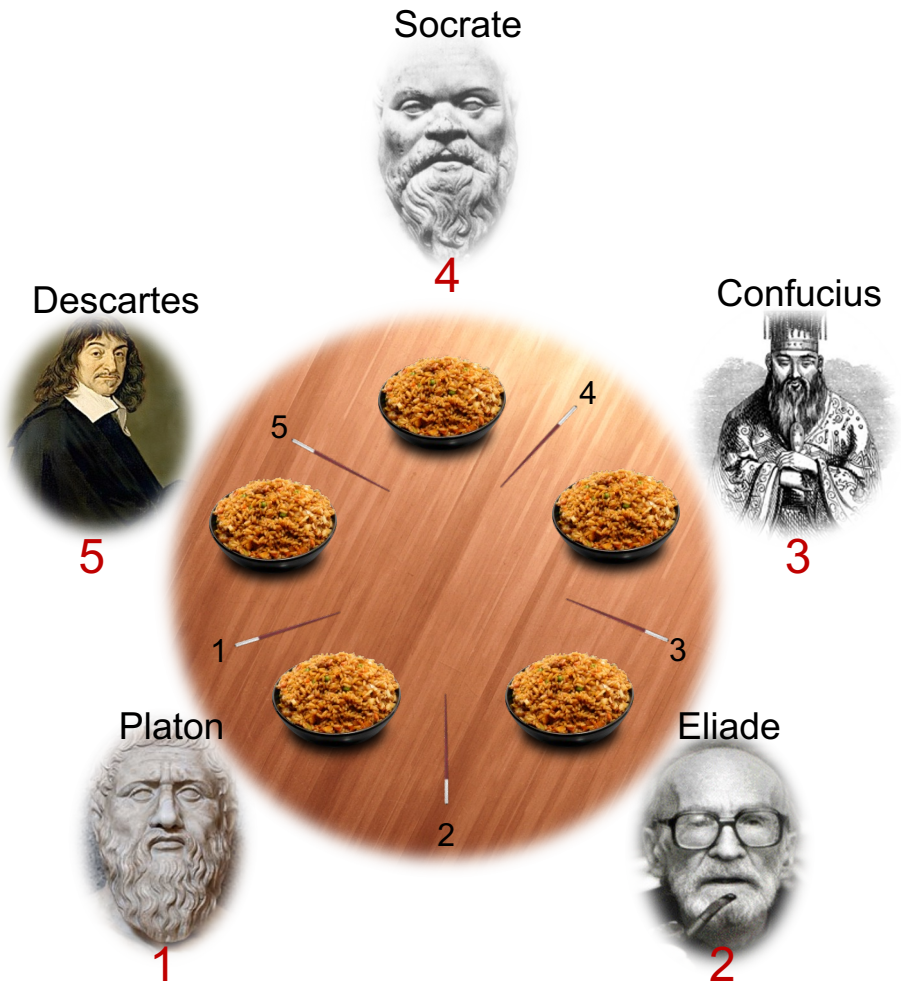
Problema filozofilor

```
process Filozof[i=1 to 5]{
    while (true) {
      ia bețișoare;
      mănâncă;
      eliberează bețișoare;
      gândește;
    }
}
```



Problema filozofilor (2)

```
sem f[1:5] = ([5]1);
process Filozof[i=1 to 4]{
      while (true) {
        P(f[i]); P(f[i+1]);
        mănâncă;
        V(f[i]); V(f[i+1]);
        gândește;
} }
process Filozof[i=5]{
      while (true) {
        P(f[1]); P(f[5]);
        mănâncă;
        V(f[1]); V(f[5]);
        gândește;
} }
```



Problema cititori-scriitori

Problema cititorilor și scriitorilor
Excludere mutuală

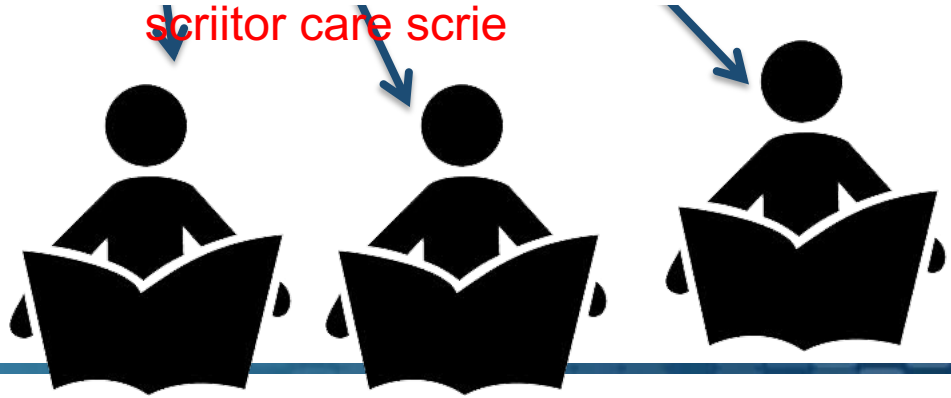
zona de memorie

(resursa critica)

1) Un singur scriitor are dreptul sa scrie la un moment dat,
2) Doar daca nu exista un cititor curent care citeste



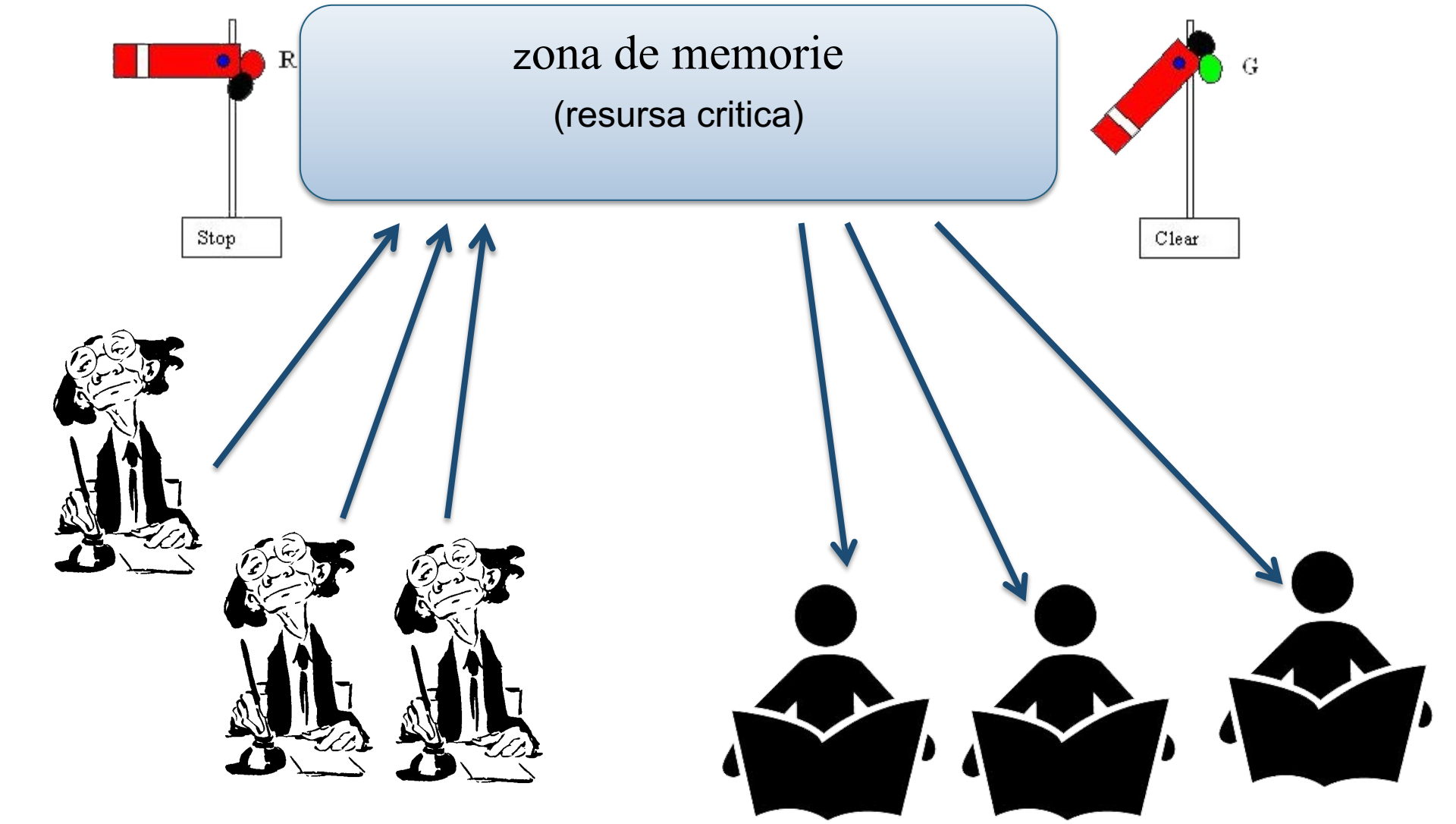
1) Un cititor poate citi din memorie, indiferent daca deja exista alti cititori ce deja citesc
2) Doar daca nu exista un scriitor care scrie



Problema cititorilor și scriitorilor
Excludere mutuală

```
sem rw = 1;
process Cititor[i=1 to m]{
      while (true){
         P(rw);
         citește din resursa comună;
         V(rw);
}    }
process Scriitor[j=1 to n]{
      while (true){
         P(rw);
         scrie în resursa comună;
         V(rw);
}    }
```




R
zona de memorie
(resursa critica)
G
Stop
Clear

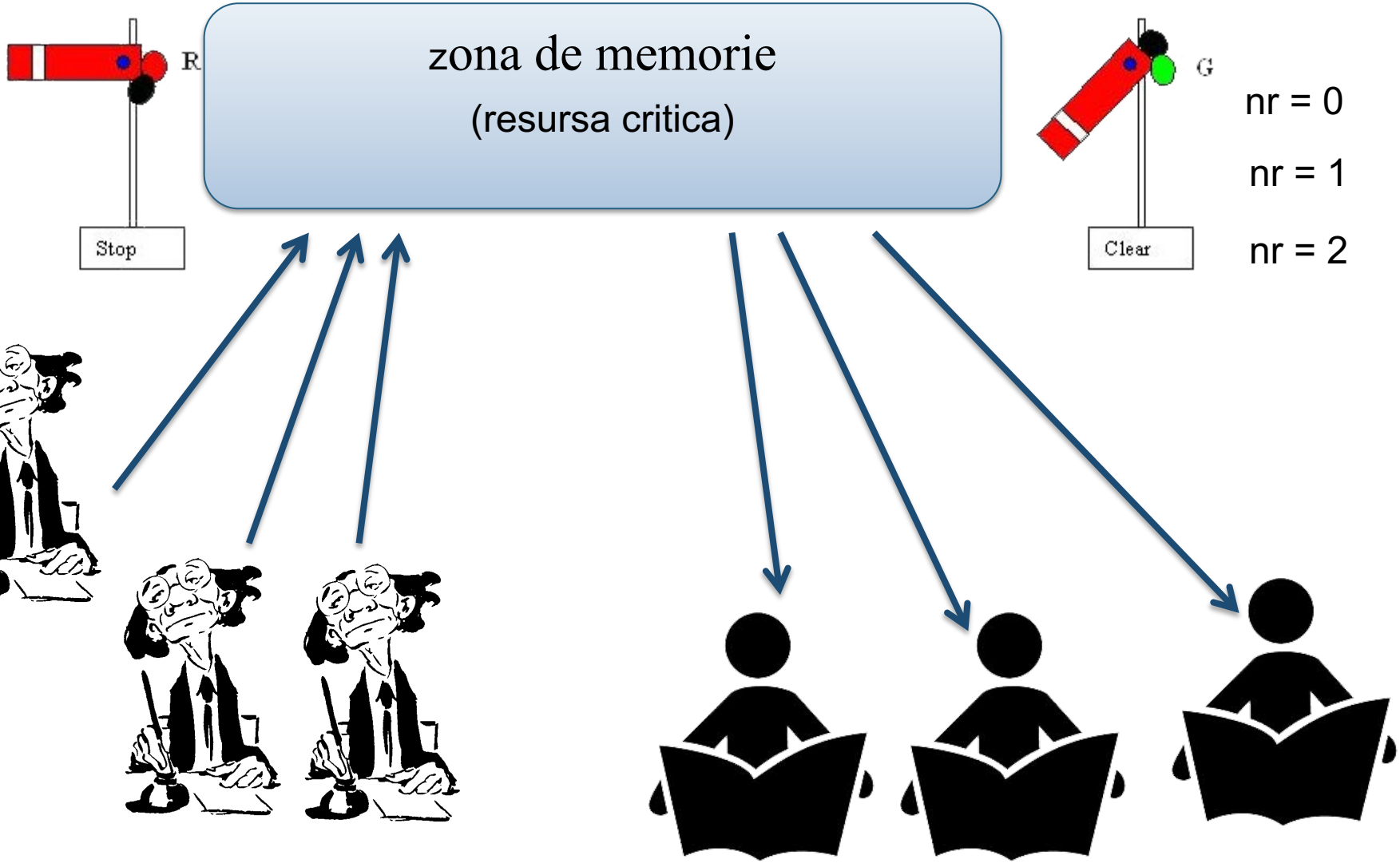
Problema cititorilor si scriitorilor
Excludere mutuală (2)

```
int nr = 0; sem mutexR = 1, rw = 1;
process Cititor[i=1 to m]{
      while (true){
        P(mutexR);
          nr = nr + 1;
          if (nr == 1) P(rw);    /* dacă primul cititor */
        V(mutexR);
        citește din resursa comună;
        P(mutexR);
          nr = nr - 1;
          if (nr == 0) V(rw);    /* dacă ultimul cititor */
        V(mutexR);
}     }
process Scriitor[j=1 to n]{
      while (true){
        P(rw);
        scrie în resursa comună;
        V(rw);
}     }
```

instructiune cu garda

sectiune critica

Kahoot!


R
Stop
zona de memorie
(resursa critica)
G
Clear
nr = 0
nr = 1
nr = 2

Problema cititorilor și scriitorilor
Sincronizare condiționată

- Invariant global:

```
RW: ( nr == 0 || nw == 0 ) && nw <= 1
```

- Proces Reader:
  - Incrementarea `nr` e condiționată de `(nw == 0)`
- Proces Writer:
  - Incrementarea `nw` e condiționată de `(nr == 0 && nw == 0)`
- Decrementarea nu trebuie condiționată

Problema cititorilor și scriitorilor
Sincronizare condiționată (2)

```
int nr = 0, nw = 0;
process Cititor[i=1 to m]{
      while (true){
        <await (nw == 0) nr = nr + 1>
        citește din resursa comună;
        <nr = nr - 1>
}     }
process Scriitor[j=1 to n]{
      while (true){
        <await (nr == 0 && nw == 0) nw = nw + 1>
        scrie în resursa comună;
        <nw = nw - 1>
}     }
```

Problema cititorilor și scriitorilor Sincronizare condiționată (3)

Politici:

- noile cereri de la cititori sunt întârziate dacă un scriitor așteaptă
- un cititor întârziat e trezit doar dacă nu există un scriitor în așteptare.

```
int nr = 0;   /* nr. cititori care folosesc resursa */
int nw = 0;   /* nr. scriitori care folosesc resursa */
sem e = 1;    /* intrare secțiune atomică */
sem r = 0;    /* intarzie cititori daca nw>0 sau dw>0 */
sem w = 0;    /* intarzie scriitori daca nw>0 sau nr>0 */
int dr = 0;   /* nr. cititori întârziați */
int dw = 0;   /* nr. scriitori întârziați */
```

Problema cititorilor și scriitorilor
Sincronizare condiționată (4)

Split binary semaphore

- Folosit pentru a implementa atât excluderea mutuală cât și sincronizarea condiționată.
- Semafoarele e, r și w formează împreună un semafor ***splitat*** *(split binary semaphore)*:
  - cel mult un semafor este 1 la un moment dat $0 \leq e + r + w \leq 1$
  - fiecare cale de execuție începe cu un P și se termină cu un singur V
  - instrucțiunile între P și V se execută în excludere mutuală.
- Tehnica se numește ***pasarea ștafetei***:
  - Inițial un semafor este 1 și un proces poate prelua ștafeta printr-o operație P asupra semaforului
  - când un proces deține ștafeta (se execută într-o secțiune critică și toate semafoarele sunt 0), el poate pasa ștafeta altui proces printr-o operație V asupra unuia din cele trei semafoare.

Problema cititorilor și scriitorilor
Sincronizare condiționată (5)

```
process Cititor[i=1 to m]{
   while (true){
     P(e);
     if (nw > 0 or dw > 0){
       dr = dr + 1; V(e); P(r);
     }
     nr = nr + 1;
     if (dr > 0) { dr = dr - 1; V(r); }
     else if (dr == 0) V(e);
     citește din resursa comună;
     P(e);
     nr = nr - 1;
     if (nr == 0 and dw > 0) { dw = dw - 1; V(w); }
     else if (nr > 0 or dw == 0) V(e);
   }
}
```

Problema cititorilor și scriitorilor
Sincronizare condiționată (6)

```
process Scriitor[j=1 to n]{
  while (true){
    P(e);
    if (nr > 0 or nw > 0) {
      dw = dw + 1; V(e); P(w)
    }
    nw = nw + 1;
    V(e);
    scrie în resursa comună;
    P(e);
    nw = nw - 1;
    if (dr > 0 and dw == 0) { dr = dr - 1; V(r); }
    else if (dw > 0) { dw = dw-1; V(w); }
    else if (dr == 0 and dw == 0) V(e);
  }
}
```

Problema barbierului

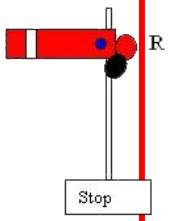
Problema bărbierului

- Problema:
  - O frizerie cu un bărbier, un scaun de bărbier, n scaune de așteptare.
  - Când nu sunt clienți, bărbierul doarme.
  - Când sosește un client fie trezește bărbierul, fie așteaptă dacă acesta e ocupat.
  - Dacă toate scaunele sunt ocupate, clientul pleacă.

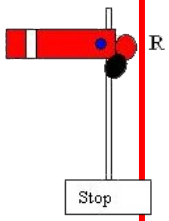


Pas 1





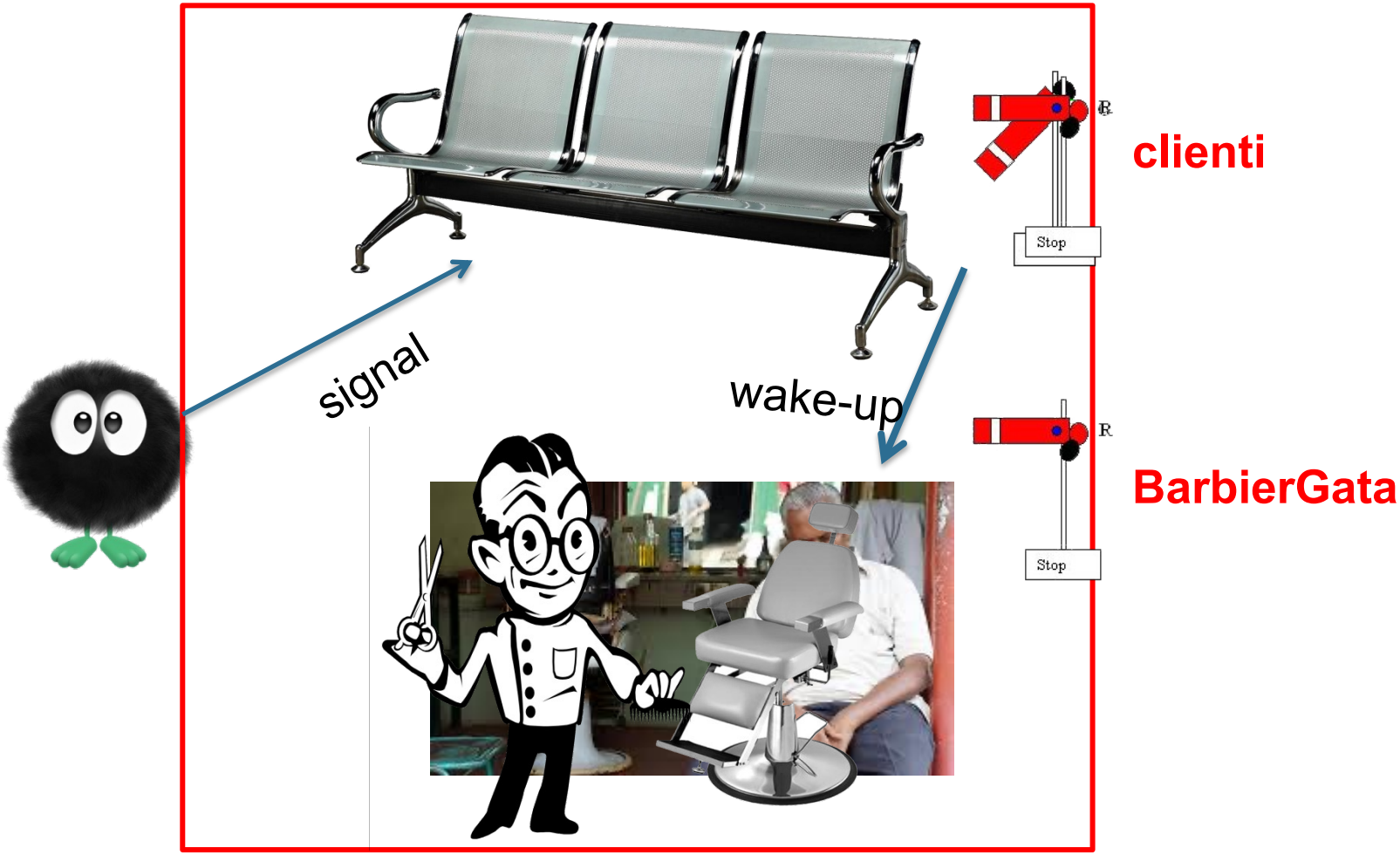
clienti



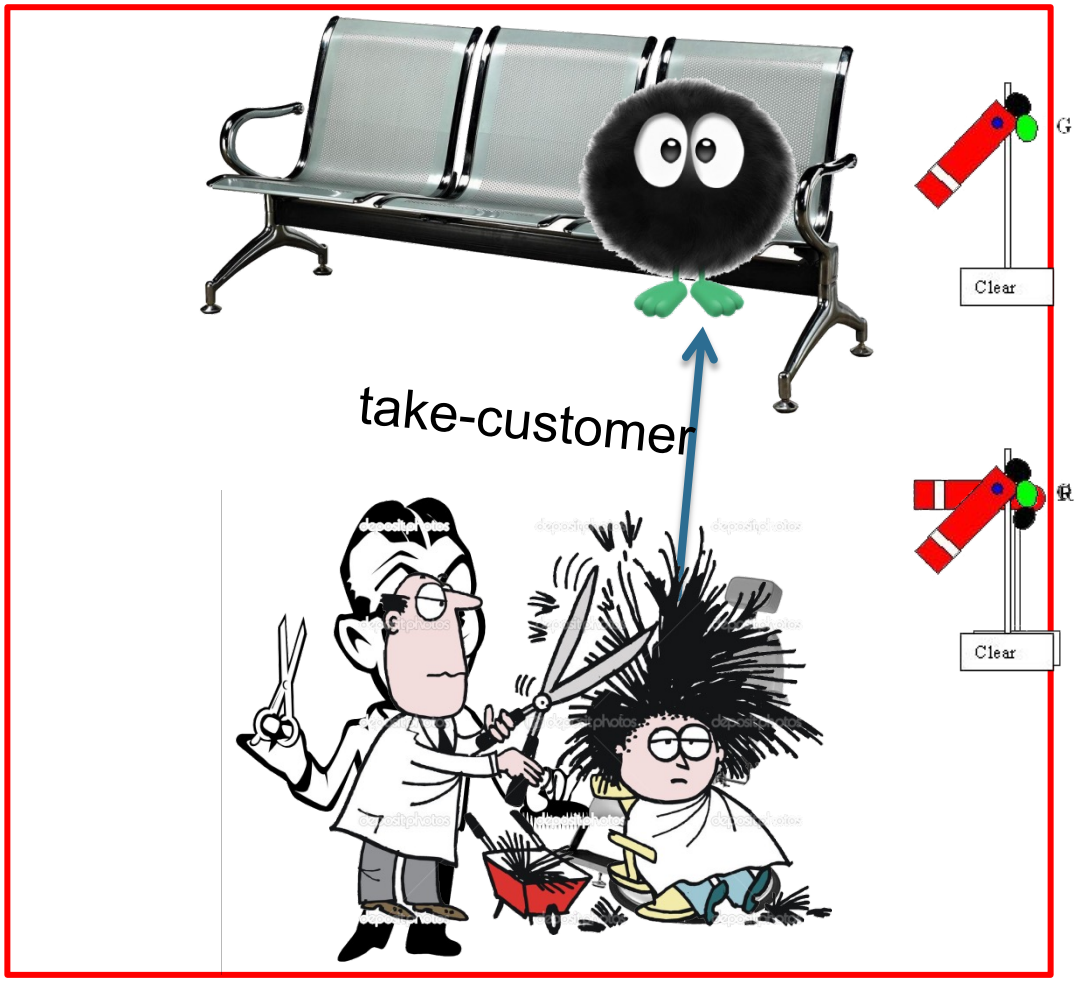
BarbierGata



Pas 2



Pas 3

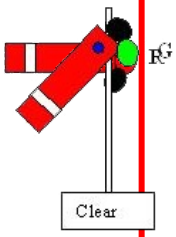


clienti

BarbierGata

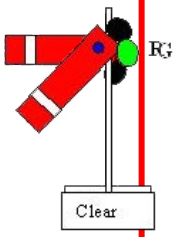
Pas 4





clienti





BarbierGata

Problema bărbierului (1)

```
int NumărScauneLibere = n;
sem Clienți = 0;
sem BărbierGata = 0;
sem Scaune = 1;
```

sincronizare cu Client

```
process Bărbier{
   while (true){
      P(Clienți);                  /* se caută un client;
                                      dacă există, este chemat */
      P(Scaune);                   /* are client, va modifica
                                      NumărScauneLibere */
      NumărScauneLibere++;         /* se eliberează un scaun */
      V(BărbierGata);              /* bărbierul e gata să tundă */
      V(Scaune);                   /* a terminat de modificat
                                      NumărScauneLibere */
      /* Bărbierul tunde... */
   }
}
```

atomicitate

Problema bărbierului (2)

```
process Client[i=1 to m]{
    while (true) {
      P(Scaune);                          /* clientul încearcă să obțină
                                             un scaun liber */
      if (NumărScauneLibere > 0){         /* există un scaun disponibil? */
        NumărScauneLibere--;              /* clientul se așază */
       V(Clienți);                        /* se anunță bărbierul că s-a
                                             ocupat un scaun */
       V(Scaune);                         /* se renunță la accesul asupra
                                             NumărScauneLibere */
       P(BărbierGata);                    /* clientul așteaptă pentru a
                                             fi tuns */
        /* Clientul e tuns... */
      } else {                            /* nu sunt scaune libere */
        V(Scaune);                        /* eliberează mutexul */
        /* Clientul pleacă netuns... */
      }
    }
}
```

sincronizare cu Barbier

atomicitate

Problema bărbierului (var. mutex)

emptyChairs = N
Clients = 0
BarberReady = 0
Chairs = 1

Bărbier

```
while(true) {
    Clients.lock();
    Chairs.lock();
    emptyChairs++;
    BarberReady.unlock();
    Chairs.unlock();
    cutHair();
}
```

Client

```
Chairs.lock();
if(emptyChairs>0) {
    emptyChairs--;
    Clients.unlock();
    Chairs.unlock();
    BarberReady.lock();
    getHairCut();
} else {
    Chairs.unlock();
}
```

Sumar

- Dezvoltarea algoritmilor folosind variabile partajate (MIMD)
- Semafoare
- Secțiuni critice
- Probleme:
  - Producători și consumatori
  - Problema filozofilor
  - Problema cititorilor și scriitorilor
  - Problema bărbierului